

Metaheurísticas

Trabajo Final: Optimización Basada en Biogeografía

Curso 2025-26



**UNIVERSIDAD
DE GRANADA**

Estudiante:

Leandro Jorge Fernández Vega

Email: leandrofdez@correo.ugr.es

Grupo de Prácticas:

Jueves - 15:30 a 17:30

Escuela Técnica Superior de Ingeniería Informática y Telecomunicaciones

Universidad de Granada

Índice

1. Introducción	3
2. Optimización Basada en Biogeografía (BBO)	3
2.1. Fundamentos Teóricos	3
2.1.1. Modelo de Equilibrio de MacArthur-Wilson	4
2.1.2. Probabilidad de Conteo de Especies en Estado Estacionario	4
2.1.3. Analogía con la Optimización	5
2.2. Algoritmo BBO Base	6
2.2.1. Tasas de Migración	6
2.2.2. Selección por Ruleta	6
2.2.3. Operador de Migración	6
2.2.4. Probabilidades de Especies y Mutación	8
2.2.5. Bucle Principal del BBO	9
2.3. BBO Mejorado (BBOImproved)	10
2.3.1. Elitismo	10
2.3.2. Migración Combinada	10
2.3.3. Mutación Gaussiana	11
2.4. Variantes Meméticas de BBO	13
2.4.1. BBOLocalSearch (Problema del Portfolio)	13
2.4.2. BBOSolisWets (Problemas Continuos Generales)	13
3. Aplicación de los Algoritmos al Problema del Portfolio	15
3.1. Descripción y Formulación del Problema	15
3.1.1. Conceptos Financieros Base	15
3.2. Consideraciones Comunes	16
3.2.1. Esquema de Representación de Soluciones	16
3.2.2. Función Objetivo	16
3.2.3. Operadores Comunes	19
3.3. Búsqueda Local (Primero el Mejor)	21
4. Aplicación de los Algoritmos al Benchmark CEC2017	23
4.1. Descripción del Benchmark	23
4.2. Consideraciones Comunes	23
4.2.1. Esquema de Representación de Soluciones	23
4.2.2. Función Objetivo	23
4.2.3. Operadores Comunes	25
4.3. Algoritmos BBO Aplicados al CEC2017	26
5. Estructura del Código y Manual de Usuario	27
6. Experimentos y Análisis de Resultados del Portfolio	30
6.1. Descripción del Entorno de Experimentación	30
6.2. Resultados Experimentales	32
6.3. Análisis del Rendimiento y Dinámica de la Familia BBO	34
6.3.1. BBO Base frente a BBOImproved	34
6.3.2. Análisis de las Hibridaciones Meméticas: Ausencia de Mejora	35
6.4. Análisis Comparativo con Otras Familias de Algoritmos	35
6.4.1. BBO frente a Técnicas de Trayectoria	35

6.4.2. BBO frente a Algoritmos Genéticos y Meméticos	35
6.5. Observaciones sobre la Fase de Test	36
7. Experimentos CEC2017: Comparación con el Estado del Arte	37
7.1. Elección de Variante BBO	37
7.2. Motivación y Selección de Algoritmos de Referencia	37
7.3. Selección de Funciones Representativas	37
7.4. Descripción del Entorno de Experimentación	38
7.5. Resultados Experimentales	38
7.6. Curvas de Convergencia	39
7.7. Análisis Comparativo	39
7.7.1. F_4 (Rastrigin): Diversidad Frente a Multimodalidad	39
7.7.2. F_5 (Scaffer's F6): Exploración en Paisajes Irregulares	40
7.7.3. F_{21} (Composición): Paisajes Heterogéneos	40
7.7.4. Escalabilidad Dimensional	40
8. Conclusiones	41
8.1. Sobre las Variantes de BBO	41
8.2. BBO en el Problema del Portfolio	41
8.3. BBO en el Benchmark CEC2017	41
8.4. Valoración Global	42

1. Introducción

Las metaheurísticas son estrategias de optimización de propósito general, diseñadas para abordar problemas donde los métodos exactos resultan inviables por la complejidad combinatoria o la ausencia de información analítica. A diferencia de los algoritmos clásicos de optimización, las metaheurísticas no garantizan la obtención del óptimo global, pero son capaces de encontrar soluciones de alta calidad en tiempos de cómputo razonables.

En este trabajo se estudia la **Optimización Basada en Biogeografía** (*Biogeography-Based Optimization*, BBO) [5], una metaheurística poblacional propuesta por Dan Simon en 2008. BBO se inspira en los modelos matemáticos de la biogeografía, la ciencia que estudia la distribución geográfica de las especies biológicas, para guiar el proceso de búsqueda. A lo largo de la memoria, se presentan tanto el algoritmo base como varias mejoras propuestas: elitismo con migración combinada y mutación gaussiana (BBOImproved), e hibridación memética con búsquedas locales específicas del dominio (BBOLocalSearch y BBOSolisWets).

Para evaluar el rendimiento de los algoritmos propuestos, se utilizan dos bancos de pruebas complementarios:

- **Problema del Portfolio:** Un problema de optimización financiera real que consiste en determinar la asignación óptima de capital entre un conjunto de activos bursátiles, maximizando el retorno esperado y minimizando el riesgo. Las soluciones están sujetas a restricciones de suma y rango que reflejan las limitaciones de una cartera de inversión real.
- **Benchmark CEC2017:** Un conjunto estándar de 30 funciones de prueba para optimización continua con restricciones de caja [1], ampliamente utilizado en la comunidad de computación evolutiva. Incluye funciones unimodales, multimodales, híbridas y de composición con dificultad creciente.

La combinación de ambos bancos de pruebas permite evaluar los algoritmos tanto en un escenario aplicado con restricciones complejas y estructura financiera real, como en un entorno académico estandarizado que facilita la comparación con otros métodos de la literatura.

2. Optimización Basada en Biogeografía (BBO)

La Optimización Basada en Biogeografía (*Biogeography-Based Optimization*, BBO) [5] es una metaheurística poblacional inspirada en los principios matemáticos de la biogeografía, la ciencia que estudia la distribución geográfica de las especies biológicas. Fue propuesta por Dan Simon en 2008 y se fundamenta en los modelos de migración de especies desarrollados por MacArthur y Wilson en su obra *The Theory of Island Biogeography* [4].

2.1. Fundamentos Teóricos

En biogeografía, un **hábitat** (o isla) es cualquier área geográficamente aislada que alberga especies biológicas. La calidad de un hábitat para sostener vida se cuantifica mediante el **Índice de Idoneidad del Hábitat** (*Habitat Suitability Index*, HSI), que depende de factores como la diversidad de vegetación, temperatura, precipitaciones o área del territorio. Las variables que caracterizan esta habitabilidad se denominan **Variables de Índice de Idoneidad** (*Suitability Index Variables*, SIVs).

2.1.1. Modelo de Equilibrio de MacArthur-Wilson

El modelo matemático de biogeografía insular de MacArthur y Wilson [4] establece que el número de especies S en un hábitat evoluciona según la ecuación diferencial:

$$\frac{dS}{dt} = \lambda(S) - \mu(S)$$

donde $\lambda(S)$ es la tasa de inmigración y $\mu(S)$ la tasa de emigración. En el modelo lineal:

$$\lambda(S) = I \left(1 - \frac{S}{S_{max}} \right), \quad \mu(S) = E \cdot \frac{S}{S_{max}}$$

siendo I la tasa máxima de inmigración, E la tasa máxima de emigración y S_{max} la capacidad máxima de especies. El punto de equilibrio S^* se obtiene igualando ambas tasas:

$$I \left(1 - \frac{S^*}{S_{max}} \right) = E \cdot \frac{S^*}{S_{max}} \implies S^* = \frac{I \cdot S_{max}}{I + E}$$

En BBO, este modelo se discretiza: cada hábitat tiene un rango $i \in \{0, 1, \dots, N-1\}$ determinado por su fitness (de peor a mejor). Las tasas de inmigración y emigración se asignan linealmente según este rango:

$$\lambda_i = 1 - \frac{i}{N}, \quad \mu_i = \frac{i}{N}$$

La propiedad fundamental es que $\lambda_i + \mu_i = 1$ para todo i . Un hábitat de rango 0 (peor fitness) tiene $\lambda_0 = 1$ (máxima inmigración) y $\mu_0 = 0$ (ninguna emigración), mientras que un hábitat de rango $N-1$ (mejor fitness) tiene $\lambda_{N-1} \approx 0$ y $\mu_{N-1} \approx 1$.

2.1.2. Probabilidad de Conteo de Especies en Estado Estacionario

La dinámica de especies en cada hábitat puede modelarse como un proceso de nacimiento-muerte (*birth-death Markov chain*) con N estados posibles $\{0, 1, \dots, N-1\}$. Sea P_s la probabilidad de que un hábitat contenga exactamente s especies en estado estacionario. La ecuación de balance detallado establece:

$$\lambda_s \cdot P_s = \mu_{s+1} \cdot P_{s+1}, \quad s = 0, 1, \dots, N-2$$

Sustituyendo las tasas lineales y resolviendo recursivamente:

$$P_{s+1} = \frac{\lambda_s}{\mu_{s+1}} \cdot P_s = \frac{1 - s/N}{(s+1)/N} \cdot P_s = \frac{N-s}{s+1} \cdot P_s$$

Desarrollando la recursión desde P_0 :

$$P_s = P_0 \cdot \prod_{k=0}^{s-1} \frac{N-k}{k+1} = P_0 \cdot \frac{N!}{s! \cdot (N-s)!} \cdot \frac{1}{N} = P_0 \cdot \binom{N-1}{s} \cdot \frac{1}{\text{cte.}}$$

Al normalizar para que $\sum_{s=0}^{N-1} P_s = 1$, se obtiene exactamente la distribución binomial simétrica con parámetro $p = 0,5$:

$$P_s = \frac{\binom{N-1}{s}}{2^{N-1}}$$

Esta distribución tiene forma de campana centrada en $s = (N - 1)/2$: los rangos intermedios son los más probables (equilibrio estable), mientras que los rangos extremos (muy buenos o muy malos) son improbables (estados transitorios). Esta propiedad es la que fundamenta el operador de mutación de BBO.

2.1.3. Analogía con la Optimización

En BBO, cada solución candidata al problema de optimización se representa como un hábitat, y cada variable de decisión como un SIV. Las buenas soluciones (alto HSI, equivalente al *fitness*) tienen alta emigración y baja inmigración, lo que significa que comparten sus características con otras soluciones. Las malas soluciones tienen alta inmigración, recibiendo características de las buenas.

A diferencia de otros algoritmos poblacionales como los Algoritmos Genéticos, las soluciones en BBO no mueren al final de cada generación: sobreviven y se adaptan a lo largo de todo el proceso de optimización. BBO comparte esta propiedad con PSO y DE, pero se diferencia en que las soluciones comparten directamente sus atributos (SIVs) mediante migración, en lugar de utilizar velocidades (PSO) o diferencias entre soluciones (DE).

La migración en BBO puede interpretarse probabilísticamente. Sea $x_i^{(j)}$ el SIV j -ésimo del hábitat i . La probabilidad de que este SIV sea reemplazado por un valor procedente de otro hábitat es λ_i . El hábitat fuente k se selecciona con probabilidad proporcional a μ_k , es decir:

$$\Pr(\text{fuente} = k) = \frac{\mu_k}{\sum_{l=0}^{N-1} \mu_l} = \frac{k/N}{\sum_{l=0}^{N-1} l/N} = \frac{2k}{N(N-1)}$$

Esta distribución sesgada hacia los hábitats de mayor rango es la que permite que la información de las buenas soluciones se propague hacia las malas, manteniendo al mismo tiempo la diversidad de la población.

2.2. Algoritmo BBO Base

El algoritmo BBO base opera sobre una población de N hábitats (soluciones). En cada generación se ejecutan dos operadores principales: migración y mutación, seguidos de una ordenación por fitness.

2.2.1. Tasas de Migración

Tras ordenar la población de peor (posición 0) a mejor (posición $N - 1$), se asignan tasas lineales de inmigración y emigración.

Algorithm 1 Cálculo de Tasas de Migración

```
1: function CALCULARTASASMIGRACION( $N$ )
2:   for  $i = 0$  to  $N - 1$  do
3:      $\lambda[i] \leftarrow 1, 0 - i/N$  ▷ Inmigración: peores reciben más
4:      $\mu[i] \leftarrow i/N$  ▷ Emigración: mejores aportan más
5:   end for
6:   return  $\lambda, \mu$ 
7: end function
```

2.2.2. Selección por Ruleta

La selección del hábitat fuente durante la migración se realiza mediante un esquema de ruleta proporcional a las tasas de emigración μ : los hábitats con mejor fitness (y por tanto mayor μ) tienen mayor probabilidad de ser seleccionados como fuente de SIVs.

Algorithm 2 Selección por Ruleta

```
1: function RULETA( $pesos$ )
2:    $total \leftarrow \sum pesos$ 
3:   if  $total \approx 0$  then
4:     return índice aleatorio uniforme en  $[0, |pesos| - 1]$ 
5:   end if
6:    $r \leftarrow \text{aleatorio\_real}(0, total)$ 
7:    $acumulado \leftarrow 0$ 
8:   for  $i = 0$  to  $|pesos| - 1$  do
9:      $acumulado \leftarrow acumulado + pesos[i]$ 
10:    if  $acumulado \geq r$  then
11:      return  $i$ 
12:    end if
13:  end for
14:  return  $|pesos| - 1$ 
15: end function
```

2.2.3. Operador de Migración

Para cada hábitat i y cada SIV j , con probabilidad λ_i se reemplaza el valor $pop[i][j]$ por el valor correspondiente de un hábitat fuente seleccionado por ruleta sobre las tasas de emigración μ . La migración opera sobre una copia de la población para evitar interferencias entre las modificaciones.

Algorithm 3 Migración BBO Base

```
1: function MIGRAR(problem, pop, fit, pop_copia, λ, μ)
2:   for  $i = 0$  to  $N_{proc} - 1$  do
3:     cambiado  $\leftarrow falso$ 
4:     for  $j = 0$  to  $D - 1$  do                                      $\triangleright D = \text{número de SIVs por hábitat}$ 
5:       if  $\text{aleatorio\_real}(0, 1) < \lambda[i]$  then
6:         fuelle  $\leftarrow \text{RULETA}(\mu)$ 
7:          $pop[i][j] \leftarrow pop\_copia[fuelle][j]$ 
8:         cambiado  $\leftarrow verdadero$ 
9:       end if
10:    end for
11:    if cambiado then
12:      REPARAR( $pop[i]$ )
13:       $fit[i] \leftarrow \text{FITNESS}(pop[i])$ 
14:    end if
15:  end for
16: end function
```

2.2.4. Probabilidades de Especies y Mutación

La mutación en BBO se basa en las probabilidades de conteo de especies en estado estacionario derivadas en la sección 2.1.2. Recordando que $P_s = \binom{N-1}{s}/2^{N-1}$, la tasa de mutación de cada hábitat se define como inversamente proporcional a su probabilidad de equilibrio:

$$m_i = m_{max} \cdot \left(1 - \frac{P_i}{P_{max}}\right)$$

donde $P_{max} = \max_s P_s = P_{\lfloor (N-1)/2 \rfloor}$ es la probabilidad del rango central (el más estable). La intuición es la siguiente:

- **Rangos intermedios** ($P_i \approx P_{max}$): $m_i \approx 0$. Estos hábitats se encuentran en un estado de equilibrio estable donde la migración ya está mejorándolos progresivamente; mutar sería contraproducente.
- **Rangos extremos** ($P_i \ll P_{max}$): $m_i \approx m_{max}$. Los hábitats muy buenos son improbables de mantener naturalmente, por lo que la mutación introduce perturbaciones exploratorias. Los hábitats muy malos necesitan perturbaciones para escapar de regiones de baja calidad donde la migración por sí sola podría ser insuficiente.

Las probabilidades se calculan eficientemente usando la recursión de los coeficientes binomiales:

Algorithm 4 Probabilidades de Especies

```

1: function PROBABILIDADES_ESPECIES( $N$ )
2:    $P[0] \leftarrow 1,0$ 
3:   for  $k = 1$  to  $N - 1$  do
4:      $P[k] \leftarrow P[k - 1] \cdot (N - k)/k$  ▷ Coeficiente binomial  $\binom{N-1}{k}$ 
5:   end for
6:   Normalizar  $P$  para que  $\sum P[k] = 1$ 
7:   return  $P$ 
8: end function

```

Este esquema hace que los hábitats extremos (muy buenos o muy malos) tengan tasas de mutación altas, incentivando la exploración en soluciones improbables. Los hábitats intermedios, que ya se encuentran en proceso de mejora por migración, reciben tasas de mutación bajas. En la mutación base, cada SIV seleccionado se reemplaza por un valor uniforme en todo el dominio:

Algorithm 5 Mutación BBO Base

```
1: function MUTAR(problem, pop, fit, P, Pmax, dominio)
2:   for  $i = 0$  to  $N_{proc} - 1$  do
3:      $m\_rate \leftarrow m_{max} \cdot (1 - P[i]/P_{max})$ 
4:      $mutado \leftarrow falso$ 
5:     for  $j = 0$  to  $D - 1$  do
6:       if  $aleatorio\_real(0, 1) < m\_rate$  then
7:          $pop[i][j] \leftarrow aleatorio\_real(dominio_{min}, dominio_{max})$ 
8:          $mutado \leftarrow verdadero$ 
9:       end if
10:    end for
11:    if  $mutado$  then
12:      REPARAR( $pop[i]$ )
13:       $fit[i] \leftarrow FITNESS(pop[i])$ 
14:    end if
15:  end for
16: end function
```

2.2.5. Bucle Principal del BBO

El algoritmo completo inicializa la población, pre-computa las tasas de migración y las probabilidades de especies (que permanecen fijas al ser el tamaño de la población constante), y ejecuta un bucle generacional de migración, mutación y ordenación hasta agotar el presupuesto de evaluaciones. El método DESPUESDEGENERACION es un *hook* vacío en el BBO base, diseñado para ser sobrescrito por diferentes variantes.

Algorithm 6 BBO Base — Optimización Principal

```
1: function OPTIMIZARBBO(problem, max_evals) ▷ Inicialización
2:   Generar población de  $N$  hábitats aleatorios
3:   Evaluar fitness de cada hábitat;  $evals \leftarrow N$ 
4:   ORDENARPOBLACION( $pop, fit$ ) ▷ Ascendente: mejor al final
   ▷ Pre-cómputo de tasas (fijas durante toda la ejecución)
5:    $P \leftarrow PROBABILIDADES\_ESPECIES(N), P_{max} \leftarrow \max(P)$ 
6:    $\lambda, \mu \leftarrow CALCULAR\_TASAS\_MIGRACION(N)$ 
7:   while  $evals < max\_evals$  do
8:      $pop\_copia \leftarrow pop$  ▷ Snapshot previo a la migración
9:     MIGRAR( $problem, pop, fit, pop\_copia, \lambda, \mu$ )
10:    MUTAR( $problem, pop, fit, P, P_{max}, dominio$ )
11:    ORDENARPOBLACION( $pop, fit$ )
12:    DESPUESDEGENERACION( $problem, pop, fit, evals, max\_evals$ )
13:  end while
14:  return  $pop[N - 1], fit[N - 1], evals$  ▷ Mejor hábitat
15: end function
```

2.3. BBO Mejorado (BBOImproved)

La variante mejorada introduce tres modificaciones sobre el BBO base, cada una orientada a corregir una debilidad conocida del algoritmo original.

2.3.1. Elitismo

Los E mejores hábitats (por defecto $E = 2$) quedan excluidos tanto de la migración como de la mutación. Actúan como memoria persistente de las mejores soluciones descubiertas, evitando que la migración o la mutación destruyan soluciones de alta calidad. El número de hábitats procesados pasa a ser $N_{proc} = N - E$.

2.3.2. Migración Combinada

En lugar de copiar directamente el SIV de la fuente, el BBO mejorado calcula una combinación convexa entre el valor actual y el valor de la fuente [3]:

$$x_i^{(j)} \leftarrow \alpha \cdot x_{src}^{(j)} + (1 - \alpha) \cdot x_i^{(j)}, \quad \alpha \sim U(0, 1)$$

Esto preserva información parcial de ambos hábitats, produciendo descendencia más diversa mientras se sigue propagando la información de las buenas soluciones.

Análisis matemático. Sea x el valor actual del SIV y y el valor de la fuente. Tras la migración directa del BBO base, el nuevo valor es simplemente $x' = y$: se destruye toda la información del hábitat receptor. En la migración combinada:

$$x' = \alpha y + (1 - \alpha)x, \quad \alpha \sim U(0, 1)$$

La esperanza y varianza del nuevo valor son:

$$\mathbb{E}[x'] = \mathbb{E}[\alpha] \cdot y + (1 - \mathbb{E}[\alpha]) \cdot x = \frac{x + y}{2}$$

$$\text{Var}(x') = (y - x)^2 \cdot \text{Var}(\alpha) = \frac{(y - x)^2}{12}$$

El valor resultante se distribuye uniformemente en el intervalo $[\min(x, y), \max(x, y)]$. A diferencia de la copia directa (que siempre produce y), la combinación convexa genera un nuevo valor intermedio que no existía previamente en la población, aumentando la diversidad genotípica sin perder la dirección de mejora proporcionada por la fuente de alta calidad.

Algorithm 7 Migración Combinada — BBOImproved

```
1: function MIGRARCOMBINADA(problem, pop, fit, pop_copia, λ, μ)
2:   for  $i = 0$  to  $N - E - 1$  do ▷ Excluir hábitats élite
3:     cambiado  $\leftarrow$  falso
4:     for  $j = 0$  to  $D - 1$  do
5:       if aleatorio_real(0, 1) <  $\lambda[i]$  then
6:         fuelle  $\leftarrow$  RULETA( $\mu$ )
7:          $\alpha \leftarrow$  aleatorio_real(0, 1)
8:          $pop[i][j] \leftarrow \alpha \cdot pop\_copia[fuelle][j] + (1 - \alpha) \cdot pop[i][j]$ 
9:         cambiado  $\leftarrow$  verdadero
10:      end if
11:    end for
12:    if cambiado then
13:      REPARAR( $pop[i]$ )
14:       $fit[i] \leftarrow$  FITNESS( $pop[i]$ )
15:    end if
16:  end for
17: end function
```

2.3.3. Mutación Gaussiana

En lugar de reemplazar un SIV por un valor aleatorio uniforme en todo el dominio, se aplica una perturbación gaussiana centrada en el valor actual [2]:

$$x_i^{(j)} \leftarrow x_i^{(j)} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2), \quad \sigma = \sigma_g \cdot (b - a)$$

donde $[a, b]$ es el dominio permitido y σ_g es una fracción configurable del rango (por defecto 0,1). El resultado se acota al dominio: $x_i^{(j)} \leftarrow \max(a, \min(b, x_i^{(j)}))$.

Análisis matemático. La diferencia fundamental entre ambos operadores reside en el desplazamiento esperado respecto al valor actual. Para la mutación uniforme del BBO base, el nuevo valor $x' \sim U(a, b)$ produce un desplazamiento medio de:

$$\mathbb{E}[|x' - x|] = \frac{(b - a)}{3} + \frac{(x - a)(b - x)}{(b - a)}$$

Para la mutación gaussiana con $\sigma = 0,1(b - a)$, el desplazamiento esperado es:

$$\mathbb{E}[|\varepsilon|] = \sigma \sqrt{\frac{2}{\pi}} \approx 0,08 \cdot (b - a)$$

Es decir, la perturbación gaussiana produce desplazamientos del orden de un 8 % del rango del dominio, frente al ~ 33 % de la mutación uniforme. Además, la probabilidad de que la perturbación gaussiana exceda 2σ es solo del 4,6 % (regla empírica), lo que concentra la exploración en el vecindario local del valor actual. Este operador transforma la mutación de un mecanismo de **exploración global** (uniforme) a uno de **explotación local** (gaussiana), favoreciendo el refinamiento progresivo de las soluciones.

Algorithm 8 Mutación Gaussiana — BBOImproved

```
1: function MUTARGAUSSIANA(problem, pop, fit, P, Pmax, dominio, σg)
2:    $\sigma \leftarrow \sigma_g \cdot (\text{dominio}_{\text{max}} - \text{dominio}_{\text{min}})$ 
3:   for  $i = 0$  to  $N - E - 1$  do ▷ Exclure habitats élite
4:      $m_{\text{rate}} \leftarrow m_{\text{max}} \cdot (1 - P[i]/P_{\text{max}})$ 
5:      $\text{mutado} \leftarrow \text{falso}$ 
6:     for  $j = 0$  to  $D - 1$  do
7:       if  $\text{aleatorio\_real}(0, 1) < m_{\text{rate}}$  then
8:          $\text{pop}[i][j] \leftarrow \text{pop}[i][j] + \mathcal{N}(0, \sigma)$ 
9:          $\text{pop}[i][j] \leftarrow \text{máx}(\text{dominio}_{\text{min}}, \text{mín}(\text{dominio}_{\text{max}}, \text{pop}[i][j]))$ 
10:         $\text{mutado} \leftarrow \text{verdadero}$ 
11:       end if
12:     end for
13:     if  $\text{mutado}$  then
14:       REPARAR( $\text{pop}[i]$ )
15:        $\text{fit}[i] \leftarrow \text{FITNESS}(\text{pop}[i])$ 
16:     end if
17:   end for
18: end function
```

2.4. Variantes Meméticas de BBO

Las variantes meméticas combinan la exploración global del BBO con la explotación local de un algoritmo de búsqueda local. Ambas variantes se implementan mediante el *hook* DESPUES-DEGENERACION: cada cierto número de generaciones, se aplica una búsqueda local al mejor hábitat de la población. Si la búsqueda local produce una mejora, el hábitat se actualiza in situ (reemplazo lamarckiano).

2.4.1. BBOLocalSearch (Problema del Portfolio)

Cada G_{LS} generaciones, aplica la Búsqueda Local (Primero el Mejor) descrita en la sección 3.3 al mejor hábitat ($pop[N-1]$). Esta variante utiliza movimientos de transferencia de capital entre activos, por lo que es específica para el problema del Portfolio.

Algorithm 9 Hook Post-Generación — BBOLocalSearch

```

1: function DESPUESDEGENERACION(problem, pop, fit, evals, max_evals)
2:    $gen \leftarrow gen + 1$ 
3:   if  $gen \bmod G_{LS} = 0$  then
4:      $presupuesto \leftarrow \min(E_{LS}, max\_evals - evals)$ 
5:     if  $presupuesto > 0$  then
6:        $resultado \leftarrow \text{BUSQUEDALOCAL}(pop[N-1], fit[N-1], presupuesto)$ 
7:        $evals \leftarrow evals + resultado.evaluaciones$ 
8:       if  $resultado.fitness > fit[N-1]$  then
9:          $pop[N-1] \leftarrow resultado.solucion$ 
10:         $fit[N-1] \leftarrow resultado.fitness$ 
11:      end if
12:    end if
13:  end if
14: end function

```

2.4.2. BBOSolisWets (Problemas Continuos Generales)

Reemplaza la Búsqueda Local del Portfolio por el método Solis-Wets, un algoritmo de búsqueda local agnóstico al dominio basado en perturbaciones aleatorias con memoria de dirección y tamaño de paso adaptativo. Esta variante es aplicable a cualquier problema continuo, incluyendo tanto el Portfolio como las funciones del benchmark CEC2017.

Búsqueda Local Solis-Wets El algoritmo de Solis y Wets [6] genera en cada iteración una perturbación aleatoria $dif[i] \sim U(0, \delta)$ y evalúa dos movimientos candidatos desde la solución actual s :

1. **Paso positivo:** $s' = s + dif + sesgo$
2. **Paso negativo** (solo si el positivo falla): $s' = s - dif - sesgo$

El vector de sesgo (*bias*) recuerda las direcciones exitosas anteriores:

- Si el paso positivo mejora: $sesgo \leftarrow 0,2 \cdot sesgo + 0,4 \cdot (dif + sesgo)$
- Si el paso negativo mejora: $sesgo \leftarrow sesgo - 0,4 \cdot (dif + sesgo)$

- Si ambos fallan: $sesgo \leftarrow sesgo/2$

El tamaño de paso δ se adapta automáticamente:

- Se **duplica** tras 5 éxitos consecutivos (explotar dirección prometedora).
- Se **reduce a la mitad** tras 3 fallos consecutivos (refinar alrededor del punto actual).

Algorithm 10 Búsqueda Local Solis-Wets

```

1: function SOLISWETS(problem, s, fs, max_evals)
2:    $sesgo \leftarrow \vec{0}, \quad \delta \leftarrow \delta_0$ 
3:    $n_{ex} \leftarrow 0, \quad n_{fa} \leftarrow 0, \quad evals \leftarrow 0$ 
4:   while  $evals < max\_evals$  do
5:     for  $i = 0$  to  $D - 1$  do
6:        $dif[i] \leftarrow \text{aleatorio\_real}(0, \delta)$ 
7:        $s'[i] \leftarrow s[i] + dif[i] + sesgo[i]$ 
8:     end for
9:     REPARAR( $s'$ )
10:     $f' \leftarrow \text{FITNESS}(s'), \quad evals \leftarrow evals + 1$ 
11:    if  $f' > f_s$  then
12:       $s \leftarrow s', \quad f_s \leftarrow f'$ 
13:       $sesgo \leftarrow 0,2 \cdot sesgo + 0,4 \cdot (dif + sesgo)$  ▷ Refuerzo positivo
14:       $n_{ex} \leftarrow n_{ex} + 1, \quad n_{fa} \leftarrow 0$ 
15:    else if  $evals < max\_evals$  then ▷ Intentar dirección opuesta
16:      for  $i = 0$  to  $D - 1$  do
17:         $s'[i] \leftarrow s[i] - dif[i] - sesgo[i]$ 
18:      end for
19:      REPARAR( $s'$ )
20:       $f' \leftarrow \text{FITNESS}(s'), \quad evals \leftarrow evals + 1$ 
21:      if  $f' > f_s$  then
22:         $s \leftarrow s', \quad f_s \leftarrow f'$ 
23:         $sesgo \leftarrow sesgo - 0,4 \cdot (dif + sesgo)$  ▷ Refuerzo negativo
24:         $n_{ex} \leftarrow n_{ex} + 1, \quad n_{fa} \leftarrow 0$ 
25:      else
26:         $sesgo \leftarrow sesgo/2$  ▷ Reducir sesgo
27:         $n_{ex} \leftarrow 0, \quad n_{fa} \leftarrow n_{fa} + 1$ 
28:      end if
29:    end if
30:    if  $n_{ex} \geq 5$  then  $n_{ex} \leftarrow 0, \quad \delta \leftarrow 2\delta$  ▷ Paso más agresivo
31:    end if
32:    if  $n_{fa} \geq 3$  then  $n_{fa} \leftarrow 0, \quad \delta \leftarrow \delta/2$  ▷ Paso más fino
33:    end if
34:  end while
35:  return  $s, f_s, evals$ 
36: end function

```

Algorithm 11 Hook Post-Generación — BBOSolisWets

```
1: function DESPUESDEGENERACION(problem, pop, fit, evals, max_evals)
2:   gen  $\leftarrow$  gen + 1
3:   if gen mód  $G_{SW}$  = 0 then
4:     presupuesto  $\leftarrow$  mín( $E_{SW}$ , max_evals - evals)
5:     if presupuesto > 0 then
6:       resultado  $\leftarrow$  SOLISWETS(problem, pop[N - 1], fit[N - 1], presupuesto)
7:       evals  $\leftarrow$  evals + resultado.evaluaciones
8:       if resultado.fitness > fit[N - 1] then
9:         pop[N - 1]  $\leftarrow$  resultado.solucion
10:        fit[N - 1]  $\leftarrow$  resultado.fitness
11:      end if
12:    end if
13:  end if
14: end function
```

3. Aplicación de los Algoritmos al Problema del Portfolio

3.1. Descripción y Formulación del Problema

El Problema del Portfolio es un desafío fundamental en el mundo de las finanzas que busca determinar la asignación óptima de un capital entre N activos disponibles. El objetivo principal de este proceso de decisión es maximizar el retorno esperado de la inversión, al mismo tiempo que se intenta minimizar el riesgo asociado y se garantiza una correcta diversificación de la cartera.

Aunque en la literatura existen múltiples versiones más complejas de este problema (que pueden incorporar valores fijos de acciones, límites en el número total de compañías a invertir, topes de riesgo máximo tolerado, costes por transacción u optimización dinámica continua), en este trabajo se aborda un modelo estático y acotado.

En este enfoque particular, el problema de optimización consiste en determinar el porcentaje exacto de capital que se debe invertir en cada una de las acciones disponibles en el mercado. Para asegurar una solución realista, esta asignación debe respetar ciertas restricciones sobre los porcentajes mínimos y máximos permitidos por cada acción. Además, el modelo asume que dichos porcentajes, una vez calculados y asignados por el algoritmo, se mantendrán fijos durante todo el periodo de inversión evaluado.

3.1.1. Conceptos Financieros Base

Para evaluar cualquier cartera, partimos de una matriz de datos donde cada celda representa un retorno obtenido μ_{ti} , calculado a partir del valor diario de la acción. Un valor positivo indica que ha habido mejora, y un valor negativo indica que se ha perdido dinero ese día.

A partir de estos datos, y siendo T el número de días, se define estadísticamente:

- **Media por acción:** $\bar{\mu}_i = \left(\sum_{t=1}^T \mu_{ti} \right) / T$
- **Matriz de Covarianza** (Riesgo entre pares de acciones): $\Sigma_{ij} = \frac{1}{T} \sum_{t=1}^T (\mu_{ti} - \bar{\mu}_i) \cdot (\mu_{tj} - \bar{\mu}_j)$

3.2. Consideraciones Comunes

3.2.1. Esquema de Representación de Soluciones

Buscamos un vector de pesos $w \in \mathbb{R}^N$ donde cada w_i representa la fracción de capital invertido en el activo i . Esta representación está sujeta a las siguientes restricciones:

1. **Presupuesto Total:** $\sum_{i=1}^N w_i = 1$
2. **Rango de Inversión:** $w_i \in [lo, hi] \cup \{0\}$ $lo, hi \in \mathbb{R}$

3.2.2. Función Objetivo

La calidad de una solución (vector de pesos) se mide mediante la función de *fitness*, la cual maximiza el beneficio pero penaliza el riesgo mediante un factor de aversión $\lambda = 500$:

$$fitness(w) = Beneficio(w) - \lambda \cdot Riesgo(w)$$

- **Beneficio Acumulado:** $Beneficio(w) = \sum_{i=1}^N w_i \left(\sum_{t=1}^T \log(1 + \mu_{ti}) \right)$
- **Riesgo:** $Riesgo(w) = \sqrt{\sum_{i,j=1}^N w_i \cdot \Sigma_{ij} \cdot w_j}$

El cálculo se realiza iterando sobre los activos y apoyándose en la matriz de covarianzas previamente calculada.

Algorithm 12 Cálculo de Media Aritmética y Suma Logarítmica

```
1: function CALCULARPROMEDIOS(datos)
2:    $T \leftarrow$  número de filas de datos
3:    $N \leftarrow$  número de columnas de datos
4:   Inicializar vector  $\mu$  de tamaño  $N$  con ceros
5:   Inicializar suma_logaritmica de tamaño  $N$  con ceros
6:   for  $t = 1$  to  $T$  do
7:     for  $i = 1$  to  $N$  do
8:        $\mu[i] \leftarrow \mu[i] + \text{datos}[t][i]$ 
9:        $\text{suma\_logaritmica}[i] \leftarrow \text{suma\_logaritmica}[i] + \log(1 + \text{datos}[t][i])$ 
10:    end for
11:  end for
12:  for  $i = 1$  to  $N$  do
13:     $\mu[i] \leftarrow \mu[i]/T$ 
14:  end for
15:  return  $\mu$ 
16: end function
```

Algorithm 13 Cálculo de la Matriz de Covarianzas

```
1: function CALCULARCOVARIANZAS(datos,  $\mu$ )
2:    $T \leftarrow$  número de filas de datos
3:   Inicializar matriz  $\Sigma$  de tamaño  $N \times N$  con ceros
4:   for  $i = 1$  to  $N$  do ▷ Se inicia en  $i$  para calcular solo la matriz triangular superior
5:     for  $j = i$  to  $N$  do
6:        $suma\_cov \leftarrow 0$ 
7:       for  $t = 1$  to  $T$  do
8:          $suma\_cov \leftarrow suma\_cov + (datos[t][i] - \mu[i]) \cdot (datos[t][j] - \mu[j])$ 
9:       end for
▷ Promediar y espejar el resultado a la matriz triangular inferior
10:       $\Sigma[i][j] \leftarrow suma\_cov / T$ 
11:       $\Sigma[j][i] \leftarrow suma\_cov / T$ 
12:    end for
13:  end for
14: end function
```

Algorithm 14 Cálculo del Beneficio Esperado

```
1: function OBTENERBENEFICIO( $w$ )
2:    $beneficio \leftarrow 0$  ▷ Multiplica el peso por la suma logarítmica histórica precalculada de cada activo
3:   for  $i = 1$  to  $N$  do
4:      $beneficio \leftarrow beneficio + w_i \cdot suma\_logaritmica[i]$ 
5:   end for
6:   return  $beneficio$ 
7: end function
```

Algorithm 15 Cálculo del Riesgo de la Cartera

```
1: function OBTENERRIESGO(solucion)
2:    $N \leftarrow$  tamaño de solucion
3:    $riesgo \leftarrow 0$ 
4:   for  $i = 1$  to  $N$  do ▷ Suma de la varianza del activo  $i$ 
5:      $riesgo \leftarrow riesgo + solucion[i] \cdot solucion[i] \cdot matriz\_covarianzas[i][i]$ 
▷ Suma de la covarianza cruzada (aprovechando matriz simétrica)
6:     for  $j = i + 1$  to  $N$  do
7:        $riesgo \leftarrow riesgo + 2 \cdot solucion[i] \cdot solucion[j] \cdot matriz\_covarianzas[i][j]$ 
8:     end for
9:   end for
10:  return  $\sqrt{riesgo}$ 
11: end function
```

Algorithm 16 Cálculo de la Función Objetivo (Fitness)

```
1: function EVALUARFITNESS( $w$ )  
2:    $beneficio \leftarrow$  OBTENERBENEFICIO( $w$ )  
3:    $riesgo \leftarrow$  CALCULARRIESGO( $w$ )  
4:    $fitness \leftarrow beneficio - \lambda \cdot riesgo$   
5:   return  $fitness$   
6: end function
```

▷ 1. Asignación del beneficio esperado

▷ 2. Cálculo del riesgo del portfolio

▷ 3. Penalización por riesgo ($\lambda = 500$)

3.2.3. Operadores Comunes

Operador de Reparación Debido a las estrictas restricciones de rango y suma, cualquier cambio aleatorio o movimiento de búsqueda suele generar soluciones inválidas. El proceso de reparación sigue esta lógica:

Algorithm 17 Operador de Reparación de Soluciones (Parte 1)

```
1: function REPARAR( $w$ )
2:    $suma \leftarrow \sum w_i$ 
3:   if  $suma = 0$  then ▷ Evitar división por cero
4:      $w_1 \leftarrow 1$ 
5:      $suma \leftarrow 1$ 
6:   end if
7:   ▷ 1. Normalización inicial
8:   for  $i = 1$  to  $N$  do
9:      $w_i \leftarrow w_i / suma$ 
10:  end for
11:  ▷ 2. Forzar límites y contar activos válidos
12:   $activos \leftarrow 0$ 
13:  for  $i = 1$  to  $N$  do
14:    if  $w_i < lo$  then
15:       $w_i \leftarrow 0$ 
16:    else
17:      if  $w_i > hi$  then  $w_i \leftarrow hi$ 
18:      end if
19:       $activos \leftarrow activos + 1$ 
20:    end if
21:  end for
22:  ▷ 3. Asegurar viabilidad matemática mínima
23:   $min\_req \leftarrow \lceil 1/hi \rceil$ 
24:  if  $activos < min\_req$  then
25:    for  $i = 1$  to  $N$  mientras  $activos < min\_req$  do
26:      if  $w_i = 0$  then
27:         $w_i \leftarrow lo$ 
28:         $activos \leftarrow activos + 1$ 
29:      end if
30:    end for
31:  end if
```

Algorithm 18 Operador de Reparación de Soluciones (Parte 2)

```
32:                                     ▷ 4. Algoritmo de redistribución iterativa
33:    $suma \leftarrow \sum w_i$ 
34:    $dif \leftarrow 1,0 - suma$ 
35:    $posible\_repartir \leftarrow verdadero$ 
36:   while  $|dif| > 0$  y  $posible\_repartir$  do
37:      $candidatos \leftarrow \emptyset$ 
38:     for  $i = 1$  to  $N$  do
39:       if  $(dif > 0 \text{ y } 0 < w_i < hi)$  o  $(dif < 0 \text{ y } w_i > lo)$  then
40:         Añadir  $i$  a  $candidatos$ 
41:       end if
42:     end for
43:     if  $|candidatos| > 0$  then
44:        $inc \leftarrow dif / |candidatos|$ 
45:        $dif \leftarrow 0$                                      ▷ Se acumularán los nuevos excesos o defectos
46:       for cada  $i \in candidatos$  do
47:          $w_i \leftarrow w_i + inc$ 
48:         if  $w_i > hi$  then
49:            $dif \leftarrow dif + (w_i - hi)$ 
50:            $w_i \leftarrow hi$ 
51:         else if  $w_i < lo$  then
52:            $dif \leftarrow dif + (w_i - lo)$ 
53:            $w_i \leftarrow lo$ 
54:         end if
55:       end for
56:     else
57:        $posible\_repartir \leftarrow falso$ 
58:     end if
59:   end while
60:   return  $w$ 
61: end function
```

Generación de Soluciones Aleatorias Para iniciar los algoritmos (o generar la población de la búsqueda aleatoria), se construyen soluciones desde cero escogiendo aleatoriamente qué activos reciben inversión (respetando los límites) y cuáles se descartan (peso cero), para luego normalizar.

Algorithm 19 Generación Aleatoria de una Solución

```
1: function CREARSOLUCION
2:    $w \leftarrow$  vector de  $N$  valores aleatorios en  $[lo, hi]$ 
3:    $n\_ceros \leftarrow$  aleatorio( $1, N - 1$ )                                     ▷ Cantidad de activos a anular
4:    $indices \leftarrow$  barajar( $[1, 2, \dots, N]$ )                               ▷ Selección sin reemplazo
5:   for  $i = 1$  to  $n\_ceros$  do
6:      $w[indices[i]] \leftarrow 0$                                              ▷ Desactivar activos seleccionados
7:   end for
8:    $w \leftarrow$  REPARAR( $w$ )                                                 ▷ Asegurar restricciones
9:   return  $w$ 
10: end function
```

3.3. Búsqueda Local (Primero el Mejor)

La Búsqueda Local implementada sigue una estrategia de trayectoria partiendo de una solución inicial generada de forma aleatoria. Para navegar por el espacio de búsqueda, se utiliza un esquema de actualización "Primero el Mejor", lo que significa que el algoritmo actualiza la solución actual en el instante exacto en que encuentra un vecino que mejora estrictamente el *fitness*, reiniciando la exploración del entorno desde la nueva posición.

Definición del Entorno y Exploración El vecindario de una solución se define teóricamente como el conjunto de todos los pares posibles de activos (i, j) donde $i \neq j$.

Operador de Movimiento El movimiento o transición hacia una solución vecina consiste en una transferencia directa de capital entre dos activos. Específicamente, se extrae un porcentaje denominado *ratio*, fijado en un 40 % según las recomendaciones de la literatura, del peso actual del activo origen i , y se le suma al peso del activo destino j .

Matemáticamente, la operación de cambio genera un vector w' donde:

$$w'_i = (1 - \text{ratio}) \cdot w_i$$

$$w'_j = w_j + (\text{ratio} \cdot w_i)$$

Dado que esta sustracción e incremento puede violar los límites individuales de inversión (lo, hi) de los activos implicados, la nueva solución candidata w' siempre es construida cumpliendo las restricciones.

Criterio de Parada El proceso iterativo concluye bajo dos condiciones: cuando se alcanza un óptimo local (es decir, se explora por completo la lista de pares desordenada sin encontrar ninguna solución que mejore el *fitness* actual) o cuando se alcanza el número máximo de evaluaciones permitidas.

Algorithm 20 Búsqueda Local (Primero el Mejor) con Inyección de Solución

```
1: function BUSQUEDALOCAL(max_evals)
2:   inicial  $\leftarrow$  CREARSOLUCION
3:   fit_inicial  $\leftarrow$  EVALUARFITNESS(inicial)
4:   return OPTIMIZARTRAYECTORIA(inicial, fit_inicial, max_evals)
5: end function

6: function OPTIMIZARTRAYECTORIA(current, fitness, max_evals)
7:    $\triangleright$  Lógica central de optimización
8:   best  $\leftarrow$  current, best_fit  $\leftarrow$  fitness, evals  $\leftarrow$  0
9:   P  $\leftarrow$   $\{(i, j) \mid i, j \in \{1 \dots N\}, i \neq j\}$   $\triangleright$  Entorno de pares posibles
10:  mejora  $\leftarrow$  verdadero
11:  ratio_t  $\leftarrow$  ratio  $\triangleright$  Cast escalar precalculado fuera del bucle
12:  umbral  $\leftarrow$  lo  $- \varepsilon$   $\triangleright$  Límite mínimo activo precalculado fuera del bucle
13:  while mejora y evals  $<$  max_evals do
14:    mejora  $\leftarrow$  falso
15:    inicio  $\leftarrow$  aleatorio(0,  $|P| - 1$ )  $\triangleright$  Punto de partida aleatorio con recorrido circular
16:    for k = 0 to  $|P| - 1$  mientras  $\neg$ mejora y evals  $<$  max_evals do
17:      (i, j)  $\leftarrow$  P[(inicio + k) mód  $|P|$ ]
18:      if best[i]  $>$  0 then
19:        mov  $\leftarrow$  ratio_t  $\cdot$  best[i]
20:        if best[j]  $<$   $\varepsilon$  y mov  $<$  lo then mov  $\leftarrow$  lo
21:        end if
22:        mov  $\leftarrow$  mín(mov, hi  $-$  best[j])  $\triangleright$  Límite superior destino
23:        mov  $\leftarrow$  mín(mov, best[i]  $-$  lo)  $\triangleright$  Límite inferior origen
24:        valido  $\leftarrow$   $\neg$ (mov  $\approx$  0 o (best[j]  $<$   $\varepsilon$  y mov  $<$  umbral))
25:        if valido then
26:          orig_i  $\leftarrow$  best[i], orig_j  $\leftarrow$  best[j]
27:          best[i]  $\leftarrow$  best[i]  $-$  mov, best[j]  $\leftarrow$  best[j]  $+$  mov
28:          fit_vec  $\leftarrow$  EVALUARFITNESS(best), evals  $\leftarrow$  evals  $+$  1
29:          if fit_vec  $>$  best_fit then  $\triangleright$  First Improvement
30:            best_fit  $\leftarrow$  fit_vec, mejora  $\leftarrow$  verdadero
31:          else
32:            best[i]  $\leftarrow$  orig_i, best[j]  $\leftarrow$  orig_j  $\triangleright$  Revertir movimiento
33:          end if
34:        end if
35:      end if
36:    end for
37:  end while
38:  return best, best_fit, evals
39: end function
```

4. Aplicación de los Algoritmos al Benchmark CEC2017

4.1. Descripción del Benchmark

El benchmark CEC2017 [1] es un conjunto estándar de funciones de prueba para optimización de variable continua con restricciones de caja, ampliamente utilizado en la comunidad de computación evolutiva para evaluar y comparar metaheurísticas. El conjunto consta de 30 funciones (F_1 a F_{30}), clasificadas en cuatro categorías según su complejidad:

- **Funciones unimodales** ($F_1 - F_3$): Poseen un único mínimo global sin óptimos locales engañosos. Evalúan la capacidad de convergencia del algoritmo.
- **Funciones multimodales simples** ($F_4 - F_{10}$): Presentan múltiples óptimos locales. Evalúan la capacidad del algoritmo para evitar estancamiento prematuro.
- **Funciones híbridas** ($F_{11} - F_{20}$): Combinan múltiples funciones base aplicadas a subconjuntos de las variables de decisión, creando paisajes de fitness heterogéneos.
- **Funciones de composición** ($F_{21} - F_{30}$): Componen varias funciones base con diferentes transformaciones y pesos, generando los paisajes más complejos y engañosos.

Todas las funciones están definidas en el dominio $[-100, 100]^D$, donde D es la dimensionalidad del problema. El mínimo global de cada función F_k se sitúa en $f(x^*) = k \times 100$. Las dimensiones válidas soportadas por la librería son $D \in \{2, 10, 20, 30, 50, 100\}$.

4.2. Consideraciones Comunes

4.2.1. Esquema de Representación de Soluciones

Una solución es un vector real $x \in \mathbb{R}^D$ donde cada componente x_i representa una variable de decisión acotada:

$$x_i \in [-B, B] \quad \forall i \in \{1, \dots, D\}$$

donde B es el semirango del dominio de búsqueda (por defecto $B = 100$).

A diferencia del problema del Portfolio, no existen restricciones de suma total ni relaciones entre componentes: cada variable es completamente independiente.

4.2.2. Función Objetivo

La calidad de una solución se mide mediante la función de fitness del benchmark CEC2017. Dado que el framework de metaheurísticas implementado maximiza la función objetivo, el valor crudo devuelto por el benchmark se niega:

$$fitness(x) = -f_{CEC}(x)$$

De esta forma, maximizar el fitness equivale a minimizar la función de benchmark original. El error respecto al óptimo global se calcula como:

$$error = f_{CEC}(x) - f(x^*) = f_{CEC}(x) - k \times 100$$

Algorithm 21 Cálculo de la Función Objetivo — CEC2017

```
1: function EVALUARFITNESS( $x$ )  
2:    $f_{CEC} \leftarrow \text{CEC17FITNESS}(x)$  ▷ Llamada a la librería CEC2017  
3:   return  $-f_{CEC}$  ▷ Negación para maximización  
4: end function
```

4.2.3. Operadores Comunes

Operador de Reparación La reparación en el dominio CEC2017 se reduce a un acotamiento (*clamping*) de cada variable al rango permitido. Esta operación es mucho más sencilla que la reparación del Portfolio, ya que no existen restricciones de suma ni límites de activación/desactivación de activos.

Algorithm 22 Reparación por Acotamiento — CEC2017

```
1: function REPARAR( $x$ )
2:   for  $i = 1$  to  $D$  do
3:      $x[i] \leftarrow \text{máx}(-B, \text{mín}(B, x[i]))$ 
4:   end for
5:   return  $x$ 
6: end function
```

Generación de Soluciones Aleatorias La creación de soluciones iniciales consiste en muestrear cada componente de forma uniforme e independiente en el dominio.

Algorithm 23 Generación Aleatoria — CEC2017

```
1: function CREARSOLUCION
2:    $x \leftarrow$  vector de  $D$  componentes
3:   for  $i = 1$  to  $D$  do
4:      $x[i] \leftarrow \text{aleatorio\_real}(-B, B)$ 
5:   end for
6:   return  $x$ 
7: end function
```

Verificación de Validez Una solución es válida si todas sus componentes se encuentran dentro del dominio.

Algorithm 24 Verificación de Validez — CEC2017

```
1: function ESVALIDA( $x$ )
2:   for  $i = 1$  to  $D$  do
3:     if  $x[i] < -B$  o  $x[i] > B$  then
4:       return falso
5:     end if
6:   end for
7:   return verdadero
8: end function
```

4.3. Algoritmos BBO Aplicados al CEC2017

Para el benchmark CEC2017, se aplica la variante **BBOImproved** descrita en la sección anterior, que combina elitismo, migración combinada y mutación gaussiana. A diferencia del problema del Portfolio, los operadores no requieren adaptación específica al dominio: la reparación se limita al acotamiento de variables y la generación aleatoria es uniforme en $[-B, B]^D$.

La variante **BBOSolisWets** es también directamente aplicable al CEC2017, ya que la búsqueda local Solis-Wets es agnóstica al dominio: solo requiere la función de fitness y el operador de reparación por acotamiento.

La siguiente tabla resume la correspondencia entre los operadores del Portfolio y los del CEC2017:

Cuadro 1: Correspondencia de operadores entre Portfolio y CEC2017

Operador	Portfolio	CEC2017
Representación	Vector de pesos $w \in [lo, hi]^N \cup \{0\}^N$	Vector real $x \in [-B, B]^D$
Restricciones	$\sum w_i = 1$, rangos $[lo, hi]$	Solo caja: $[-B, B]$
Reparación	Normalización + redistribución iterativa	Acotamiento (<i>clamping</i>)
Fitness	$Beneficio - \lambda \cdot Riesgo$	$-f_{CEC}(x)$
BL memética	Transferencia de capital (BBOLocalSearch)	Solis-Wets (BBOSolisWets)

5. Estructura del Código y Manual de Usuario

Estructura del Proyecto El código fuente se ha organizado siguiendo un patrón modular dentro de la carpeta **software/**, la cual actúa como directorio raíz. El árbol de directorios resultante es el siguiente:

- **src/**: Directorio principal de código fuente. Contiene los ficheros C++ (**.cpp**), las fuentes C del benchmark CEC2017 (**cec17.c**, **cec17_test_func.c**) y el archivo principal **main.cpp**. Se subdivide en:
 - **common/**: Clases base y utilidades del *framework* de metaheurísticas (**mh.h**, **mhtrayectory.h**, **problem.h**, **solution.h**, **util.h**).
 - **inc/**: Cabeceras (**.h**) específicas de esta práctica. Incluye los problemas (**portfolio.h**, **cec2017problem.h**), los algoritmos base (**localsearch.h**, **soliswets.h**, **randomsearch.h**, **greedy.h**) y las variantes de BBO (**bbo.h**, **bbo_improved.h**, **bbo_local_search.h**, **bbo_soliswets.h**).
 - **others/**: *Scripts* y utilidades adicionales (generación de *boxplots* en Python).
- **datos_portfolio_2526/**: Ficheros de datos históricos en formato CSV. Contiene los conjuntos de entrenamiento y test para cada índice bursátil:
 - **ibex_35.csv** / **ibex_35_2025.csv**: IBEX 35 (Train / Test).
 - **syp_100.csv** / **syp_100_2025.csv**: S&P 100 (Train / Test).
 - **syp_500.csv** / **syp_500_2025.csv**: S&P 500 (Train / Test).
- **input_data/**: Datos auxiliares requeridos por la librería CEC2017. Contiene las matrices de rotación (**M_kDd.txt**) y los vectores de desplazamiento (**shift_data_k.txt**) para cada función k y dimensión d .
- **results/**: Directorio de salida para los resultados del modo Portfolio (archivos CSV).
- **results_cec17/**: Directorio de salida para los resultados del modo CEC2017.
- **Directorio raíz (software/)**: Contiene el ejecutable generado (**main**), el archivo de parámetros (**config.txt**) y los ficheros de configuración del sistema de construcción (**CMakeLists.txt** y **Makefile**).

Manual de Compilación Para facilitar la portabilidad y la compilación del proyecto, se ha utilizado el sistema automático **CMake**. Posicionados en la carpeta raíz **software/**, se ejecutan los siguientes comandos:

1. Generar los archivos de construcción:

```
cmake .
```

2. Compilar el proyecto utilizando el **Makefile** generado:

```
make
```

Ejemplo de Ejecución El programa soporta dos modos de operación seleccionables mediante el parámetro `mode` del fichero de configuración:

- `mode=portfolio`: Ejecuta los algoritmos sobre los *datasets* financieros. Los resultados se guardan en `results/`.
- `mode=cec2017`: Ejecuta los algoritmos BBO sobre las funciones del benchmark CEC2017. Los resultados se guardan en `results_cec17/`.

Para iniciar las pruebas, se ejecuta por consola:

```
./main (<ruta_fichero_parámetros>)
```

Si no se especifica ningún argumento, el programa utilizará unos parámetros por defecto. Si se provee la ruta a un fichero válido, el programa cargará su configuración. Por consola se mostrará el progreso indicando el *dataset* o función en curso, el algoritmo y el número de ejecución actual.

Configuración de Parámetros El fichero `config.txt` centraliza todos los parámetros de ambos modos de ejecución sin necesidad de recompilar el código fuente. Permite establecer el modo de operación, los parámetros generales (semilla, evaluaciones máximas, número de ejecuciones), los parámetros específicos del Portfolio (factor de aversión al riesgo λ , ratio de la BL, hiperparámetros de los algoritmos genéticos, meméticos, ES, BMB e ILS), los parámetros de BBO (tamaño de población, elitismo, sigma de mutación gaussiana, frecuencia y presupuesto de las búsquedas locales) y los parámetros del benchmark CEC2017 (funciones a evaluar, dimensiones, dominio). Su estructura es la siguiente:

```
# Configuration file
# Set mode=portfolio or mode=cec2017 to choose the experiment.
mode=portfolio

# Common parameters
seed=42
num_runs=50

# =====
# PORTFOLIO parameters (mode=portfolio)
# =====
max_evals=10000
lambda=500.0
ls_ratio=0.4
data_folder=./datos_portfolio_2526/

# Genetic / Memetic algorithm parameters
ga_pop_size=100
agg_pc=0.8
ga_pm=0.1
ga_tourney_k=3
ma_bl_ratio=0.15
ma_bl_maxevals=100
```

```

ma_gen_period=10
ma_rand_prob=0.1
ma_top_ratio=0.1

# Simulated Annealing (ES) parameters
es_mu=0.2
es_phi=0.3
es_tf=0.001

# BMB parameters
bmb_restarts=5
bmb_evals_per_restart=2000

# ILS parameters
ils_mut_ratio=0.2
ils_num_iters=5
ils_evals_per_iter=2000

# BBO shared parameters
bbo_elitism=2
bbo_imp_sigma=0.1

# BBOLocalSearch parameters (portfolio only)
bbo_ls_ratio=0.4
bbo_ls_every=5
bbo_ls_maxevals=200

# BBOSolisWets parameters
sw_delta=0.2
sw_every=5
sw_max_evals=200

# Datasets (format: train,test,min_weight,max_weight)
dataset=ibex_35.csv,ibex_35_2025.csv,0.005,0.08
dataset=syp_100.csv,syp_100_2025.csv,0.005,0.05
dataset=syp_500.csv,syp_500_2025.csv,0.005,0.02

# =====
# CEC2017 parameters (mode=cec2017)
# =====
bbo_pop_size=50
bbo_mutation=0.01
cec_bound=100.0
cec_funcs=1,2,3,4,5,6,7,8,9,10,11,12,13,14,
           15,16,17,18,19,20,21,22,23,24,25,26,
           27,28,29,30
cec_dims=10,30,50
cec_runs=10

```

6. Experimentos y Análisis de Resultados del Portfolio

6.1. Descripción del Entorno de Experimentación

Para evaluar el rendimiento de los algoritmos implementados, disponemos de datos históricos (2015-2024) para calcular la rentabilidad y el riesgo durante la optimización (fase de *Train*), que luego son validados de forma objetiva con datos del año 2025 (fase de *Test*). Se organizan en tres *datasets* principales:

- **IBEX35:** Empresas del IBEX 35 (españolas), un total de 30 empresas durante 2813 días.
- **SYP100:** Empresas del S&P 100 (índice de 100 mayores empresas de EE.UU.), un total de 97 empresas durante 2765 días.
- **SYP500:** Empresas del S&P 500 (índice de 500 mayores empresas de EE.UU.), un total de 457 empresas durante 2765 días.

Para garantizar la significancia estadística de los resultados empíricos, se han realizado 50 ejecuciones independientes por cada algoritmo y conjunto de datos. En cada ejecución se ha utilizado una semilla diferente para el generador de números aleatorios, comenzando en la semilla 42 y auto-incrementándose en cada iteración (*Run 1: 42, Run 2: 43, ..., Run 50: 91*).

Los parámetros de parada y configuración general han sido fijados de la siguiente manera para todos los experimentos:

- **Factor de aversión al riesgo:** $\lambda = 500$
- **Límites de inversión:**
 - **IBEX35:** $lo = 0,005$ $hi = 0,08$
 - **SYP100:** $lo = 0,005$ $hi = 0,05$
 - **SYP500:** $lo = 0,005$ $hi = 0,02$
- **Condición de parada:** Un máximo de 10,000 evaluaciones de la función objetivo (*max_evals*).
- **Búsqueda Local:** Ratio de transferencia de capital fijado en 0,4 (40 %).
- **Parámetros Comunes de los Algoritmos Genéticos:**
 - **Tamaño de la población:** $M = 100$ cromosomas.
 - **Selección:** Torneo con tamaño $k = 3$.
 - **Probabilidad de cruce (P_c):** 0,8 en el AGG y 1,0 (siempre) en el AGE.
 - **Probabilidad de mutación (P_m):** 0,1 por individuo.
 - **Parámetro de cruce BLX- α :** $\alpha = 0,3$.
 - **Operador de cruce:** BLX- α u Operador Aritmético.

- **Parámetros Específicos de los Algoritmos Meméticos (AM):**
 - **Búsqueda Local Suave:** Ratio de transferencia reducido al 0,15 (15 %).
 - **Frecuencia de aplicación:** Cada 10 generaciones.
 - **Condición de parada de la BL:** Máximo de 100 evaluaciones por individuo.
 - **AM-Rand:** Probabilidad de aplicar la BL a cada cromosoma $P_{LS} = 0,1$.
 - **AM-Best:** Fracción de la población seleccionada para la BL igual a 0,1 (el 10 % de los mejores individuos).
- **Parámetros del Enfriamiento Simulado (ES):**
 - **Ratio de empeoramiento:** $\mu = 0,2$.
 - **Probabilidad de aceptación inicial:** $\phi = 0,3$.
 - **Temperatura final:** $T_f = 10^{-3}$.
- **Parámetros de la Búsqueda Multiarranque Básica (BMB):**
 - **Número de reinicios:** 5 soluciones iniciales independientes.
 - **Evaluaciones por reinicio:** 2000 (incluyendo la evaluación de la solución inicial).
- **Parámetros de la Búsqueda Local Reiterada (ILS-BL e ILS-ES):**
 - **Ratio de mutación:** $\alpha_{mut} = 0,2$ (el 20 % de los genes se permutan).
 - **Número de iteraciones:** 5 (una inicial aleatoria y 4 sobre soluciones mutadas).
 - **Evaluaciones por iteración:** 2000 por llamada al optimizador interno.
- **Parámetros compartidos de BBO (BBOImproved, BBOLocalSearch, BBOSolisWets):**
 - **Tamaño de la población:** $N = 100$ hábitats (mismo que el GA en modo Portfolio).
 - **Tasa máxima de mutación:** $m_{max} = 0,1$.
 - **Elitismo:** $E = 2$ hábitats protegidos de migración y mutación.
 - **Sigma de mutación gaussiana (BBOImproved):** $\sigma_g = 0,1$ (fracción del rango del dominio).
- **Parámetros de BBOLocalSearch (Portfolio):**
 - **Ratio de transferencia de la BL:** 0,4 (40 %).
 - **Frecuencia de aplicación:** Cada 5 generaciones BBO.
 - **Presupuesto por invocación:** 200 evaluaciones máximas.

■ **Parámetros de BBOSolisWets:**

- **Tamaño de paso inicial (δ_0):** 0,2.
- **Frecuencia de aplicación:** Cada 5 generaciones BBO.
- **Presupuesto por invocación:** 200 evaluaciones máximas.

6.2. Resultados Experimentales

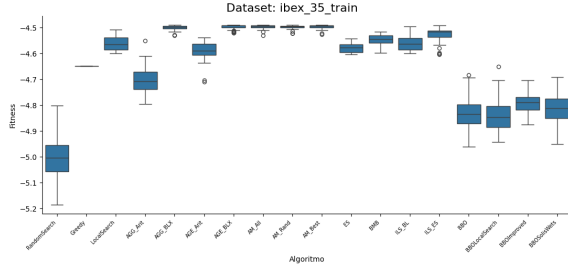
Los 18 algoritmos evaluados se agrupan en cuatro familias:

- **Baselines:** RandomSearch, Greedy, LocalSearch.
- **Algoritmos Genéticos:** AGG_Arit, AGG_BLX (Generacionales), AGE_Arit, AGE_BLX (Estacionarios), AM_All, AM_Rand, AM_Best (Meméticos).
- **Trayectorias:** ES (Enfriamiento Simulado), BMB (Búsqueda Multiarranque), ILS_BL e ILS_ES (Búsqueda Local Reiterada).
- **BBO:** BBO (base), BBOImproved, BBOLocalSearch, BBOSolisWets.

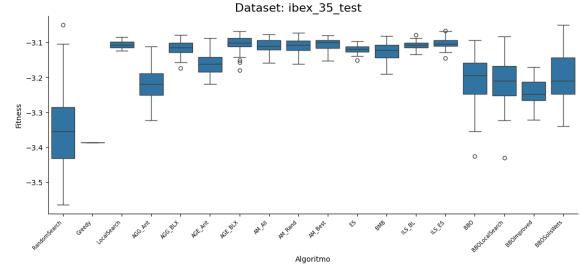
A continuación, se presentan las tablas resumen con los valores de *fitness* (mejor, peor, media y desviación típica) tanto para los datos de entrenamiento (*Train*) como para la evaluación final sobre datos no vistos (*Test*). También se incluye el rendimiento medio, el coste computacional (evaluaciones) y el tiempo de ejecución en segundos.

Cuadro 2: Resultados globales Ibex 35 (Train vs Test)

Algorithm	Best(Tr)	Worst(Tr)	Mean(Tr)	Std(Tr)	Best(Ts)	Worst(Ts)	Mean(Ts)	Std(Ts)	Return(Ts)	Evals	Time(s)
RandomSearch	-4.802	-5.186	-4.996	0.086	-3.050	-3.565	-3.349	0.117	16.414	10000	0.031
Greedy	-4.650	-4.650	-4.650	0.000	-3.387	-3.387	-3.387	0.000	29.269	1	0.000
LocalSearch	-4.508	-4.601	-4.564	0.027	-3.085	-3.124	-3.106	0.010	5.801	3888	0.002
AGG_Arit	-4.550	-4.797	-4.706	0.050	-3.111	-3.323	-3.218	0.045	14.570	10000	0.014
AGG_BLX	-4.490	-4.531	-4.500	0.008	-3.079	-3.174	-3.115	0.020	9.636	10000	0.015
AGE_Arit	-4.540	-4.711	-4.591	0.034	-3.087	-3.220	-3.163	0.029	12.690	10000	0.018
AGE_BLX	-4.490	-4.521	-4.498	0.009	-3.068	-3.179	-3.105	0.020	9.013	10000	0.014
AM_All	-4.490	-4.531	-4.498	0.007	-3.077	-3.159	-3.110	0.019	10.333	10000	0.014
AM_Rand	-4.490	-4.522	-4.498	0.007	-3.079	-3.153	-3.107	0.020	9.999	10000	0.014
AM_Best	-4.490	-4.527	-4.497	0.008	-3.079	-3.153	-3.107	0.020	10.048	10000	0.014
ES	-4.543	-4.604	-4.579	0.018	-3.097	-3.151	-3.119	0.011	5.968	10000	0.006
BMB	-4.516	-4.599	-4.546	0.019	-3.081	-3.190	-3.127	0.025	7.999	10000	0.005
ILS_BL	-4.496	-4.601	-4.562	0.027	-3.078	-3.134	-3.107	0.011	6.203	9461	0.005
ILS_ES	-4.492	-4.604	-4.529	0.028	-3.067	-3.145	-3.103	0.015	6.816	10000	0.006
BBO	-4.683	-4.962	-4.828	0.066	-3.094	-3.426	-3.204	0.069	11.196	10000	0.031
BBOLocalSearch	-4.651	-4.943	-4.842	0.063	-3.083	-3.430	-3.211	0.061	12.843	10000	0.027
BBOImproved	-4.704	-4.876	-4.791	0.037	-3.170	-3.322	-3.244	0.038	16.105	10000	0.033
BBOSolisWets	-4.691	-4.951	-4.813	0.063	-3.050	-3.340	-3.200	0.072	11.001	10000	0.029



(a) Distribución en Train

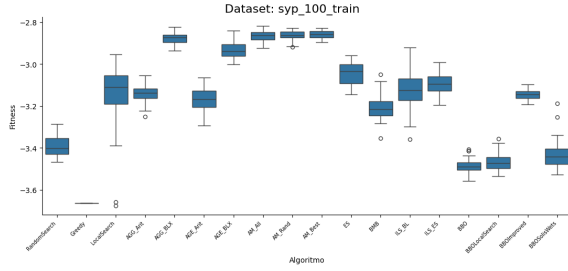


(b) Distribución en Test

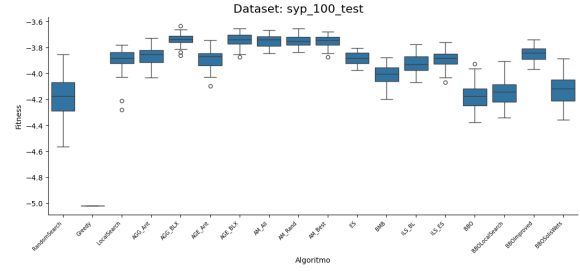
Figura 1: Boxplots comparativos del fitness de los algoritmos para el Ibex 35.

Cuadro 3: Resultados globales S&P100 (Train vs Test)

Algorithm	Best(Tr)	Worst(Tr)	Mean(Tr)	Std(Tr)	Best(Ts)	Worst(Ts)	Mean(Ts)	Std(Ts)	Return(Ts)	Evals	Time(s)
RandomSearch	-3.287	-3.467	-3.394	0.049	-3.854	-4.566	-4.186	0.168	21.815	10000	0.121
Greedy	-3.664	-3.664	-3.664	0.000	-5.020	-5.020	-5.020	0.000	2.475	1	0.000
LocalSearch	-2.953	-3.676	-3.144	0.147	-3.782	-4.281	-3.899	0.092	24.134	10000	0.052
AGG_Arit	-3.055	-3.251	-3.141	0.039	-3.730	-4.031	-3.871	0.073	21.241	10000	0.071
AGG_Blx	-2.823	-2.935	-2.878	0.023	-3.636	-3.864	-3.738	0.044	23.753	10000	0.071
AGE_Arit	-3.065	-3.295	-3.166	0.051	-3.746	-4.180	-3.887	0.077	21.827	10000	0.071
AGE_Blx	-2.841	-3.002	-2.936	0.037	-3.657	-3.873	-3.746	0.050	24.144	10000	0.076
AM_All	-2.818	-2.923	-2.863	0.024	-3.668	-3.844	-3.749	0.042	24.491	10000	0.071
AM_Rand	-2.827	-2.919	-2.863	0.021	-3.657	-3.838	-3.754	0.041	24.000	10000	0.069
AM_Best	-2.827	-2.896	-2.859	0.019	-3.679	-3.876	-3.752	0.044	24.310	10000	0.069
ES	-2.958	-3.144	-3.041	0.049	-3.805	-3.976	-3.886	0.049	25.930	10000	0.052
BMB	-3.050	-3.353	-3.207	0.056	-3.880	-4.202	-4.012	0.071	22.373	10000	0.052
ILS_BL	-2.922	-3.360	-3.127	0.087	-3.779	-4.071	-3.931	0.070	24.807	10000	0.052
ILS_ES	-2.992	-3.195	-3.092	0.047	-3.759	-4.070	-3.891	0.065	27.691	10000	0.053
BBO	-3.408	-3.559	-3.487	0.032	-3.928	-4.377	-4.177	0.108	18.670	10000	0.128
BBOLocalSearch	-3.356	-3.534	-3.467	0.041	-3.908	-4.343	-4.149	0.105	18.938	10000	0.117
BBOImproved	-3.096	-3.193	-3.147	0.025	-3.742	-3.969	-3.849	0.054	20.526	10000	0.134
BBOSolisWets	-3.187	-3.527	-3.438	0.064	-3.887	-4.356	-4.127	0.112	18.790	10000	0.121



(a) Distribución en Train

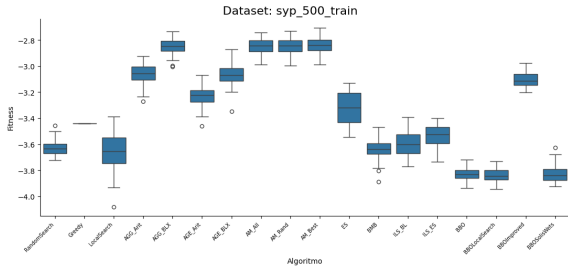


(b) Distribución en Test

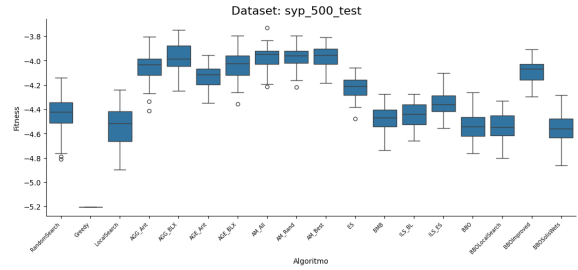
Figura 2: Boxplots comparativos del fitness de los algoritmos para el S&P100.

Cuadro 4: Resultados globales S&P500 (Train vs Test)

Algorithm	Best(Tr)	Worst(Tr)	Mean(Tr)	Std(Tr)	Best(Ts)	Worst(Ts)	Mean(Ts)	Std(Ts)	Return(Ts)	Evals	Time(s)
RandomSearch	-3.455	-3.723	-3.630	0.055	-4.140	-4.810	-4.431	0.146	6.361	10000	1.348
Greedy	-3.438	-3.438	-3.438	0.000	-5.205	-5.205	-5.205	0.000	-15.640	1	0.000
LocalSearch	-3.387	-4.082	-3.664	0.148	-4.239	-4.898	-4.539	0.156	-3.785	10000	1.116
AGG_Arit	-2.923	-3.270	-3.061	0.077	-3.803	-4.413	-4.055	0.120	3.653	10000	1.139
AGG_Blx	-2.736	-3.003	-2.851	0.062	-3.750	-4.248	-3.973	0.111	4.331	10000	1.161
AGE_Arit	-3.069	-3.459	-3.231	0.078	-3.954	-4.348	-4.134	0.098	4.016	10000	1.149
AGE_Blx	-2.872	-3.348	-3.069	0.081	-3.795	-4.358	-4.042	0.113	4.552	10000	1.176
AM_All	-2.744	-2.988	-2.845	0.062	-3.731	-4.213	-3.974	0.098	4.664	10000	1.157
AM_Rand	-2.729	-2.998	-2.845	0.065	-3.795	-4.217	-3.970	0.087	4.319	10000	1.155
AM_Best	-2.707	-2.989	-2.834	0.062	-3.807	-4.185	-3.972	0.093	4.926	10000	1.152
ES	-3.128	-3.544	-3.318	0.121	-4.059	-4.476	-4.226	0.089	-0.910	10000	1.093
BMB	-3.470	-3.889	-3.638	0.079	-4.273	-4.735	-4.478	0.101	1.621	10000	1.155
ILS_BL	-3.390	-3.771	-3.595	0.090	-4.274	-4.658	-4.444	0.101	-0.581	10000	1.125
ILS_ES	-3.398	-3.733	-3.534	0.084	-4.103	-4.557	-4.355	0.096	-1.596	10000	1.107
BBO	-3.718	-3.938	-3.830	0.048	-4.264	-4.762	-4.541	0.119	2.504	10000	1.414
BBOLocalSearch	-3.732	-3.946	-3.836	0.051	-4.331	-4.804	-4.547	0.116	3.543	10000	1.357
BBOImproved	-2.975	-3.203	-3.104	0.058	-3.906	-4.294	-4.091	0.087	2.808	10000	1.431
BBOSolisWets	-3.625	-3.923	-3.826	0.068	-4.285	-4.864	-4.561	0.122	1.166	10000	1.369



(a) Distribución en Train



(b) Distribución en Test

Figura 3: Boxplots comparativos del fitness de los algoritmos para el S&P500.

6.3. Análisis del Rendimiento y Dinámica de la Familia BBO

6.3.1. BBO Base frente a BBOImproved

El BBO base [5] presenta un rendimiento moderado y una dispersión amplia en los tres *datasets*, con una brecha respecto a los mejores algoritmos que se acentúa a medida que crece la dimensionalidad del problema. Este comportamiento se debe a su operador de migración estándar, que copia SIVs de forma directa componente a componente. En el problema del Portfolio, esta transferencia individual rompe las correlaciones entre pesos de activos que caracterizan a las carteras bien diversificadas.

BBOImproved mejora de forma consistente al BBO base en los tres *datasets*, con una diferencia que crece con la dimensionalidad. Esta mejora se sustenta en tres mecanismos: la migración combinada [3], que genera valores intermedios entre fuente y receptor mediante una combinación convexa aleatoria, preservando parte de la información del hábitat original; la mutación gaussiana [2], que concentra las perturbaciones en el vecindario local del valor actual en lugar de reemplazarlo por un valor aleatorio del dominio completo; y el elitismo, que protege a los mejores hábitats de ser modificados por migración o mutación. En los *datasets* de baja y media dimensionalidad, *BBOImproved* también reduce la dispersión de los resultados, aunque en alta dimensionalidad (S&P500) la variabilidad entre ejecuciones se mantiene en niveles similares al BBO base.

6.3.2. Análisis de las Hibridaciones Meméticas: Ausencia de Mejora

Las variantes meméticas (*BBOLocalSearch* y *BBOSolisWets*) no consiguen mejorar al BBO base en ninguno de los tres *datasets*. Ambas se sitúan en niveles de *fitness* prácticamente idénticos al BBO base, con dispersiones similares o ligeramente superiores. Este resultado contradice la expectativa habitual de que incorporar búsqueda local a una metaheurística poblacional produce mejoras, y queda claramente por debajo de *BBOImproved*.

La causa principal es un problema de asignación del presupuesto computacional. Con un límite de evaluaciones fijo, cada invocación de la búsqueda local consume evaluaciones que dejan de estar disponibles para las generaciones globales de BBO. En el caso de *BBOLocalSearch*, la búsqueda local por transferencia de capital requiere muchas evaluaciones para producir mejoras apreciables en un problema con restricciones de suma y rango, y el presupuesto asignado por invocación resulta insuficiente para generar avances significativos. En el caso de *BBOSolisWets*, el mecanismo de perturbación con paso adaptativo presenta el mismo problema: las iteraciones internas del método Solis-Wets consumen evaluaciones sin aportar mejoras que compensen la reducción del número de generaciones globales.

En ambos casos, el resultado neto es que la búsqueda local no aporta un beneficio suficiente como para justificar el coste en evaluaciones que detrae de la exploración poblacional. *BBOImproved*, que dedica la totalidad del presupuesto a las generaciones de migración y mutación mejoradas, obtiene resultados superiores con el mismo número total de evaluaciones.

6.4. Análisis Comparativo con Otras Familias de Algoritmos

6.4.1. BBO frente a Técnicas de Trayectoria

La relación entre BBO y las técnicas de trayectorias (BL, ES, BMB, ILS) depende de la dimensionalidad del problema:

- **Baja dimensionalidad (IBEX35):** Las técnicas de trayectorias superan a todas las variantes de BBO. ILS_ES e ILS_BL obtienen valores de *fitness* medios notablemente mejores que *BBOImproved*. En un espacio reducido, los mecanismos de perturbación local son capaces de explorar el vecindario de forma eficaz, y el coste de mantener una población completa no se compensa con una mejor exploración global.
- **Alta dimensionalidad (S&P500):** La situación se invierte para *BBOImproved*, que supera a todas las técnicas de trayectorias. Sin embargo, el BBO base y las variantes meméticas (*BBOLocalSearch*, *BBOSolisWets*) quedan por debajo de ES e ILS_ES. Este resultado muestra que el beneficio de la exploración poblacional en alta dimensionalidad solo se materializa cuando los operadores de BBO son adecuados (migración combinada, mutación gaussiana), no por el simple hecho de usar una población.

6.4.2. BBO frente a Algoritmos Genéticos y Meméticos

Los Algoritmos Meméticos (AM_Best, AM_All, AM_Rand) y los GA con cruce BLX (AGG_BLX, AGE_BLX) superan a todas las variantes de BBO en los tres *datasets*. La diferencia se reduce con la dimensionalidad, en el S&P500, *BBOImproved* se acerca al rendimiento de los GA generacionales con cruce aritmético (AGG_Arit), pero los AM mantienen una ventaja clara en todos los escenarios.

La razón principal es que los operadores de cruce de los GA combinan pares de soluciones

completas, preservando las correlaciones entre pesos de activos. La migración de BBO transfiere SIVs de forma individual, lo que fragmenta estas correlaciones. La BL integrada en los AM aporta además un refinamiento fino que las variantes meméticas de BBO no consiguen replicar con el presupuesto disponible.

6.5. Observaciones sobre la Fase de Test

Los resultados de *Test* permiten evaluar la capacidad de generalización de las carteras optimizadas. BBOImproved obtiene retornos positivos en los tres *datasets*, lo que indica que las carteras generadas no sufren un sobreajuste severo a los datos históricos.

En el IBEX35, BBOImproved obtiene el mejor retorno entre los algoritmos poblacionales, superando tanto a los Algoritmos Meméticos como a los GA generacionales. En el S&P100, la situación se invierte: los AM, las trayectorias múltiples y los GA con cruce BLX superan a BBOImproved.

En el S&P500, varias técnicas de trayectorias (LocalSearch, ILS y ES) presentan retornos negativos, mientras que todos los algoritmos poblacionales (GA, AM y BBO) mantienen retornos positivos. Dentro de este grupo, los GA y AM generan las carteras con mayor retorno, seguidos por las variantes de BBO. Esto sugiere que los algoritmos poblacionales generalizan mejor que las trayectorias en alta dimensionalidad, y que entre ellos los GA/AM producen carteras más robustas que BBO.

7. Experimentos CEC2017: Comparación con el Estado del Arte

7.1. Elección de Variante BBO

De las cuatro variantes implementadas, se selecciona *BBOImproved* como representante de la familia BBO para la comparación con el estado del arte. Los resultados del problema del Portfolio justifican esta elección: BBOImproved supera al BBO base en todos los *datasets*, con una ventaja que crece con la dimensionalidad. Las variantes meméticas (BBOLocalSearch y BBOSolisWets) no mejoran al BBO base debido al coste en evaluaciones de la búsqueda local, que reduce el número de generaciones globales disponibles. Además, BBOLocalSearch utiliza un operador de vecindario específico del Portfolio (transferencia de capital), lo que la hace inaplicable a funciones continuas sin restricciones de suma. BBOSolisWets es aplicable al CEC2017, pero dado que no mejora al BBO base en el Portfolio, se opta por la variante que ha demostrado el mejor rendimiento global.

7.2. Motivación y Selección de Algoritmos de Referencia

Para situar el rendimiento de BBOImproved en el contexto de la optimización continua moderna, se compara con tres algoritmos que representan el estado del arte en la competición CEC2017 [1]:

- **jSO**: Ganador de la competición CEC2017. Variante adaptativa de Evolución Diferencial con estrategia de reinicio y adaptación de parámetros. Constituye el *gold standard* de referencia.
- **ELSHADE_SPACMA**: Variante de DE adaptativa con cruce espacial. Consistentemente entre los mejores algoritmos en todas las dimensiones evaluadas.
- **EBOWithCMAR**: Enfoque de *ensemble* que combina múltiples estrategias con reinicio CMA-ES. Especialmente competitivo en funciones de composición, donde los algoritmos basados en migración suelen tener mayores dificultades.

Estos tres algoritmos representan las tres primeras posiciones de la competición y proporcionan una referencia exigente para evaluar las fortalezas y debilidades de la propuesta.

7.3. Selección de Funciones Representativas

De las 30 funciones del benchmark, se seleccionan tres que cubren las tres dimensiones críticas del rendimiento de un optimizador:

- **F_4 (Rastrigin rotada)**: Función altamente multimodal con estructura regular de óptimos locales equidistantes. Evalúa la **exploración**: la capacidad de la población BBO para mantener diversidad y evitar la convergencia prematura.
- **F_5 (Expanded Scaffer's F6 rotada)**: Función multimodal no separable que acopla pares de variables. Evalúa la **explotación con dependencias**, es decir, si la migración componente a componente de BBO puede manejar funciones donde las variables no son independientes.
- **F_{21} (Composición 1)**: Combina Rosenbrock, Elipsoidal y Rastrigin con diferentes óptimos y escalados. Evalúa la **adaptabilidad**, es decir, si la migración de BBO puede manejar

un paisaje donde distintas regiones poseen estructuras de fitness fundamentalmente diferentes.

Las dimensiones evaluadas son $D \in \{10, 30, 50\}$.

7.4. Descripción del Entorno de Experimentación

La librería CEC2017 requiere un conjunto de ficheros de datos auxiliares ubicados en el directorio `input_data/`. Estos ficheros contienen las matrices de rotación (`M_k_Dd.txt`), los vectores de desplazamiento (`shift_data_k.txt`) y las permutaciones de variables (`shuffle_data_k_Dd.txt`) necesarios para construir las funciones rotadas, desplazadas e híbridas del benchmark. Cada función k y dimensión d utiliza su propio conjunto de ficheros, lo que garantiza la reproducibilidad de los experimentos.

Los parámetros que difieren respecto a la configuración del Portfolio son los siguientes:

- **Tamaño de la población:** $N = 50$ hábitats (frente a 100 en el Portfolio).
- **Tasa máxima de mutación:** $m_{max} = 0,01$ (frente a 0,1 en el Portfolio).
- **Dominio de búsqueda:** $[-B, B]^D$ con $B = 100$.
- **Presupuesto de evaluaciones:** $10\,000 \times D$ (escalado linealmente con la dimensión).
- **Número de ejecuciones:** 10 por cada par (función, dimensión).

Los parámetros de elitismo ($E = 2$) y sigma de mutación gaussiana ($\sigma_g = 0,1$) se mantienen idénticos a los del Portfolio.

7.5. Resultados Experimentales

Las tablas siguientes recogen el error medio final ($f(x) - f(x^*)$) de cada algoritmo al agotar el 100 % del presupuesto de evaluaciones, para cada función y dimensión. Los valores de los algoritmos de referencia (jSO, ELSHADE_SPACMA, EBOwithCMAR) proceden de los resultados publicados en la competición CEC2017. El mejor resultado de cada fila aparece en **negrita**.

Cuadro 5: Error medio final de las funciones seleccionadas por dimensión. El mejor resultado de cada fila aparece en **negrita**.

Función	D	EBOwithCMAR	ELSHADE_SPACMA	BBOImproved	jSO
F_4 (Rastrigin)	10	0.00e+00	0.00e+00	6.33e+00	0.00e+00
F_4 (Rastrigin)	30	5.65e+01	0.00e+00	1.00e+02	5.87e+01
F_4 (Rastrigin)	50	4.29e+01	5.68e-01	1.29e+02	5.62e+01
F_5 (Scaffer's F6)	10	0.00e+00	1.35e+00	1.63e+01	1.76e+00
F_5 (Scaffer's F6)	30	2.78e+00	3.69e+00	1.06e+02	8.56e+00
F_5 (Scaffer's F6)	50	7.58e+00	6.22e+00	2.10e+02	1.64e+01
F_{21} (Composición)	10	1.14e+02	1.54e+02	2.15e+02	1.32e+02
F_{21} (Composición)	30	1.99e+02	2.09e+02	3.08e+02	2.09e+02
F_{21} (Composición)	50	2.11e+02	2.15e+02	4.30e+02	2.19e+02

7.6. Curvas de Convergencia

Las figuras siguientes muestran la evolución del error medio en escala logarítmica en función del porcentaje de evaluaciones consumidas. BBOImproved aparece etiquetado como la curva verde en las gráficas.

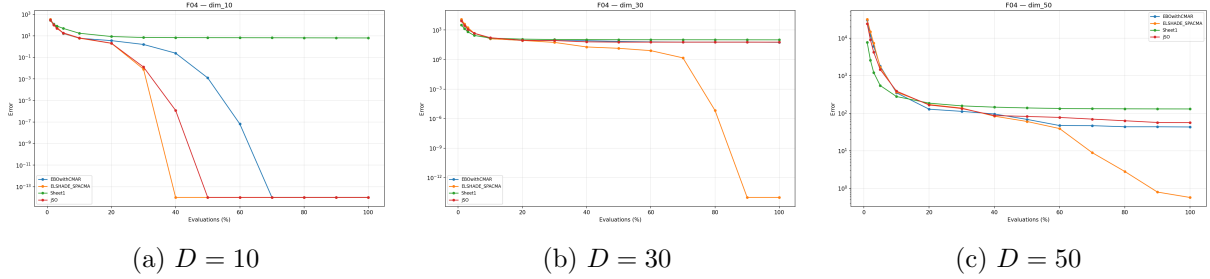


Figura 4: Convergencia de F_4 (Rastrigin) por dimensión.

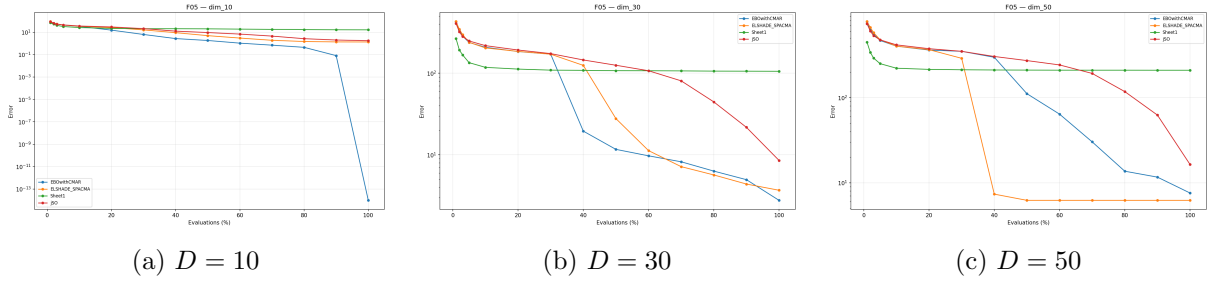


Figura 5: Convergencia de F_5 (Scaffer's F6) por dimensión.

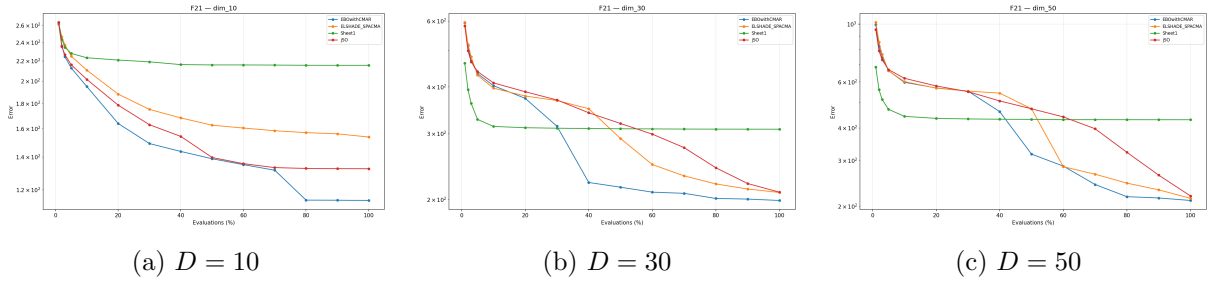


Figura 6: Convergencia de F_{21} (Composición 1) por dimensión.

7.7. Análisis Comparativo

7.7.1. F_4 (Rastrigin): Diversidad Frente a Multimodalidad

La función Rastrigin presenta un número exponencial de óptimos locales equidistantes. En $D = 10$, los tres algoritmos de referencia alcanzan el óptimo global con error cero, mientras que BBOImproved converge a un error residual moderado. La migración tiende a propagar los SIVs de las mejores soluciones hacia el resto de la población, lo que reduce la diversidad y favorece la convergencia prematura hacia la cuenca del mejor óptimo local encontrado.

Al aumentar la dimensionalidad ($D = 30, 50$), esta tendencia se acentúa: el error de BBOImproved se mantiene en el orden de 10^2 , frente a valores muy inferiores de los algoritmos de

referencia. Las curvas de convergencia muestran que BBOImproved se estanca tras consumir una fracción reducida del presupuesto, mientras que los algoritmos de referencia continúan reduciendo el error. La mutación gaussiana con $\sigma_g = 0,1$ no genera perturbaciones suficientemente amplias para saltar entre cuencas de atracción en un paisaje con periodicidad fija.

7.7.2. F_5 (Scaffer's F6): Exploración en Paisajes Irregulares

La función Expanded Scaffer's F6 combina pares de variables con una estructura no separable y asimétrica. En $D = 10$, EBOwithCMAR logra error cero y los otros algoritmos de referencia se sitúan por debajo de 2, mientras que BBOImproved queda un orden de magnitud por encima.

En $D = 30$ y $D = 50$, el error de BBOImproved se mantiene en el orden de 10^2 , frente a valores de 10^0 – 10^1 de los algoritmos de referencia. La transferencia componente a componente de la migración BBO resulta poco eficaz en una función que acopla pares de variables, ya que modificar una variable sin ajustar simultáneamente su pareja no produce mejoras. Las variantes de DE, al operar con vectores diferencia completos, preservan estas dependencias entre variables.

7.7.3. F_{21} (Composición): Paisajes Heterogéneos

La función de composición presenta el escenario más exigente para BBO. En todas las dimensiones, BBOImproved obtiene los errores más elevados de los cuatro algoritmos, con una brecha que se amplía con D .

La dificultad reside en la heterogeneidad del paisaje: distintas regiones del espacio de búsqueda están gobernadas por funciones base con propiedades diferentes (Rosenbrock, Elipsoidal, Rastrigin). La migración de BBO transfiere SIVs individuales entre soluciones, y esta transferencia componente a componente destruye las correlaciones entre variables que cada subfunción requiere. Los algoritmos de referencia operan con vectores diferencia completos (DE) o con matrices de covarianza (CMA-ES), lo que les permite preservar la estructura de correlación y adaptarse a la geometría local del paisaje.

7.7.4. Escalabilidad Dimensional

Un patrón común a las tres funciones es que el rendimiento relativo de BBOImproved se degrada al aumentar la dimensionalidad. Mientras que los algoritmos de referencia mantienen errores que crecen moderadamente con D , el error de BBOImproved se escala de forma más agresiva. Esto se debe a la naturaleza componente a componente de la migración: al tratar cada SIV de forma independiente, el algoritmo no captura las correlaciones entre variables, cuya importancia crece con la dimensionalidad del problema.

8. Conclusiones

Este trabajo ha estudiado la Optimización Basada en Biogeografía (BBO) y sus variantes, evaluándolas en dos contextos complementarios: el problema del Portfolio y el benchmark CEC2017.

8.1. Sobre las Variantes de BBO

De las cuatro variantes implementadas, BBOImproved es la que obtiene los mejores resultados de forma consistente. Las tres mejoras introducidas (elitismo, migración combinada BLX y mutación gaussiana) producen un beneficio creciente con la dimensionalidad del problema. En el Portfolio, la diferencia respecto al BBO base pasa de ser moderada en el IBEX35 (30 activos) a ser del 19 % en el S&P500 (457 activos).

Las variantes meméticas (BBOLocalSearch y BBOSolisWets) no mejoran al BBO base. La búsqueda local consume evaluaciones que dejan de estar disponibles para las generaciones globales, y con el presupuesto asignado no genera mejoras suficientes para compensar ese coste. Este resultado sugiere que, bajo presupuestos limitados, es preferible invertir todas las evaluaciones en mejorar los operadores poblacionales que dividirlos entre exploración global y explotación local.

8.2. BBO en el Problema del Portfolio

En el contexto del Portfolio, BBO presenta un perfil de rendimiento particular. En baja dimensionalidad, todas las variantes de BBO quedan por debajo de las técnicas de trayectorias (ILS, ES) y de los GA/AM. En alta dimensionalidad, BBOImproved supera a todas las técnicas de trayectorias, aunque los Algoritmos Meméticos (AM_Best) mantienen una ventaja clara en todos los escenarios.

La limitación principal de BBO en este problema es su migración componente a componente: al transferir SIVs de forma individual, fragmenta las correlaciones entre pesos de activos que caracterizan a las carteras bien diversificadas. Los GA, que combinan soluciones completas mediante cruce BLX- α , preservan mejor estas correlaciones.

En la fase de Test, todos los algoritmos poblacionales (GA, AM y BBO) mantienen retornos positivos en el S&P500, mientras que varias técnicas de trayectorias producen retornos negativos. BBOImproved obtiene el mejor retorno entre los algoritmos poblacionales en el IBEX35, aunque queda por debajo de los GA/AM en los índices de mayor dimensionalidad.

8.3. BBO en el Benchmark CEC2017

La comparación con los ganadores de la competición CEC2017 (jSO, ELSHADE_SPACMA, EBOwithCMAR) confirma que BBOImproved no alcanza el rendimiento del estado del arte en optimización continua. En las tres funciones analizadas (F_4 , F_5 , F_{21}) y en todas las dimensiones ($D = 10, 30, 50$), los algoritmos de referencia obtienen errores significativamente inferiores.

La debilidad fundamental es, de nuevo, la migración componente a componente: no captura correlaciones entre variables, lo que resulta especialmente costoso en funciones no separables (F_5) y en paisajes heterogéneos (F_{21}). Los algoritmos de referencia operan con vectores diferencia completos (DE) o matrices de covarianza (CMA-ES), lo que les permite adaptarse a la geometría local del paisaje. Además, el rendimiento relativo de BBO se degrada más rápidamente con la

dimensionalidad que el de los algoritmos de referencia.

8.4. Valoración Global

BBO es un algoritmo conceptualmente sencillo cuyo rendimiento mejora de forma apreciable con las modificaciones propuestas en BBOImproved. Su principal fortaleza es la escalabilidad frente a métodos de trayectoria en problemas de alta dimensionalidad, y su principal debilidad es la transferencia independiente de variables, que ignora las correlaciones interdimensionales.

Como líneas de trabajo futuro, las direcciones más prometedoras serían: incorporar migración por bloques de variables en lugar de componente a componente para preservar las correlaciones entre dimensiones; adaptar dinámicamente los parámetros de mutación (σ_g) y las tasas de migración durante la ejecución; y explorar presupuestos computacionales mayores para las variantes meméticas, donde la búsqueda local podría aportar un beneficio que no se observa con el presupuesto actual.

Referencias

- [1] N. H. Awad, M. Z. Ali, P. N. Suganthan, J. J. Liang, and B. Y. Qu. Problem definitions and evaluation criteria for the CEC 2017 special session and competition on single objective real-parameter numerical optimization. Technical report, Nanyang Technological University, Singapore, 2016.
- [2] Wenyin Gong, Zhihua Cai, and Charles X. Ling. A real-coded biogeography-based optimization with mutation. *Applied Mathematics and Computation*, 216(9):2749–2758, 2010.
- [3] Haiping Ma and Dan Simon. Blended biogeography-based optimization for constrained optimization. *Engineering Applications of Artificial Intelligence*, 24(3):517–525, 2011.
- [4] Robert H. MacArthur and Edward O. Wilson. *The Theory of Island Biogeography*. Princeton University Press, Princeton, NJ, 1967.
- [5] Dan Simon. Biogeography-based optimization. *IEEE Transactions on Evolutionary Computation*, 12(6):702–713, 2008.
- [6] Francisco J. Solis and Roger J.-B. Wets. Minimization by random search techniques. *Mathematics of Operations Research*, 6(1):19–30, 1981.